UT3 Bloque 4
JSP y patrón MVC

Material de estudio y guía de prácticas

Aprender a separar la coordinación de una petición de la presentación HTML

En este bloque transformarás la biblioteca de los bloques anteriores. Los Servlets seguirán recibiendo peticiones y coordinando el acceso a datos, pero dejarán de generar páginas completas con PrintWriter. Las JSP representarán la información que el controller haya preparado.

Meta de aprendizaje. Al terminar debes poder seguir un dato desde la URL o la base de datos hasta el HTML y explicar qué trabajo realiza el controller, el modelo, Tomcat y la JSP.

Práctica	Resultado
1	Primera JSP visual
2	Saludo preparado por un controller
3	Perfil dinámico
4	Ficha de un libro
5	Biblioteca con JSTL
6	Formulario dinámico
Final	Biblioteca MVC integrada

Cómo trabajar con este documento

Las prácticas parten de interfaces visualmente terminadas. Tu trabajo se concentra en los fragmentos funcionales marcados como TODO y, progresivamente, en el desarrollo de los controllers. No necesitas reconstruir el diseño ni memorizar el CSS.

Archivos proporcionados. Cada carpeta practicaXX contiene la JSP de trabajo. Los recursos comunes se encuentran en assets. Copia la vista a WEB-INF/views y los recursos a src/main/webapp/assets.

Código que debes comprender. Expression Language, JSTL, action, method, name, context path, setAttribute, getRequestDispatcher y forward.

```
src/main/java/
├─ conexion/
│  └─ ConnectionFactory.java
├─ controller/
├─ dao/
└─ model/

src/main/webapp/
├─ assets/
└─ WEB-INF/views/
```

Todos los Servlets de la unidad pertenecen a package controller. Las JSP no abren conexiones ni llaman directamente al DAO.

1 El problema de generar HTML con PrintWriter

```
PrintWriter out = response.getWriter();
out.println("<html><body>");
for (Libro libro : libros) {
    out.println("<p>" + libro.getTitulo() + "</p>");
}
out.println("</body></html>");
```

El código funciona porque escribe texto en la respuesta HTTP. El navegador interpreta ese texto como HTML. El problema es que el mismo método recibe la petición, coordina los datos y diseña la pantalla.

Problema	Consecuencia
Java y HTML mezclados	La lectura y el mantenimiento se complican.
Diseño dentro del Servlet	Cambiar una tarjeta obliga a modificar Java.
Métodos largos	La lógica HTTP queda escondida entre etiquetas.
Código repetido	Cabeceras y mensajes aparecen en varios métodos.

```
ANTES
Navegador -> Servlet -> DAO -> objetos -> PrintWriter -> HTML

OBJETIVO DEL BLOQUE
Navegador -> controller -> modelo -> controller
          -> request -> JSP -> HTML
```

2 Qué es una JSP

Una JSP es una plantilla que se procesa en el servidor. Puede contener HTML, Expression Language y etiquetas JSTL. El navegador nunca ejecuta JSP ni recibe su código fuente.

```
JSP -> Tomcat y Jasper -> HTML -> Navegador
```

Pregunta	Respuesta
¿Quién procesa la JSP?	Tomcat mediante su motor JSP.
¿Qué recibe el navegador?	HTML dentro de una respuesta HTTP.
¿Puede el navegador abrir una JSP de WEB-INF?	No, debe entrar por un controller.
¿Qué ocurriría sin Tomcat?	La plantilla no se convertiría en una respuesta dinámica.

3 Práctica 1 Primera JSP visual

Objetivo: comprobar que Tomcat procesa una JSP y que el navegador solo recibe HTML.

1. Copia assets a src/main/webapp/assets.
2. Copia practica01/bienvenida.jsp a WEB-INF/views.
3. Crea un Servlet en package controller con la ruta /bienvenida.
4. Haz que doGet entregue la petición a la vista.
5. Arranca Tomcat y abre la ruta del controller.
6. Usa Ver código fuente y busca únicamente HTML.

TODO 1 Mostrar un mensaje procesado por JSP

Completa la expresión EL de la insignia para que muestre el texto indicado en el comentario del archivo. Conserva el elemento span y su clase CSS.

Resultado esperado. Debe aparecer la pantalla completa y una insignia de confirmación. Si ves el texto ${TODO_1} o no aparece la insignia, revisa la sintaxis de EL.

```
Navegador -> GET /bienvenida -> controller
controller -> forward -> bienvenida.jsp
Tomcat -> HTML -> Navegador
```

4 Controller JSP y atributos

```
request.setAttribute("nombre", nombre);
request.getRequestDispatcher("/WEB-INF/views/saludo.jsp")
       .forward(request, response);
```

Elemento	Función
setAttribute	Añade un objeto al request con una clave.
RequestDispatcher	Selecciona un recurso interno.
forward	Continúa con los mismos request y response.

Elemento	Función
Expression Language	Lee los datos preparados sin insertar Java en la vista.

La clave utilizada en setAttribute forma un contrato entre controller y vista. Si los nombres no coinciden exactamente, la expresión no encontrará el dato.

5 Práctica 2 Saludo dinámico

Objetivo: recibir en la JSP un nombre preparado por SaludoServlet.

1. Crea SaludoServlet dentro de package controller.
2. Asocia la ruta /saludo.
3. En doGet añade un atributo llamado nombre al request.
4. Realiza forward a /WEB-INF/views/saludo.jsp.
5. Completa TODO 1 en la JSP para mostrar el atributo mediante EL.

TODO 1 Utilizar el atributo preparado

Localiza el h1 del saludo. La expresión debe solicitar el atributo cuya clave utiliza el controller. No cambies el texto, la etiqueta ni las clases visuales.

Pregunta de comprensión. ¿Cambiaría la URL visible del navegador al realizar forward? Explica la respuesta contando cuántas peticiones HTTP se producen.

6 Parámetro frente a atributo

```
Navegador -- parámetro --> controller -- atributo --> JSP
```

Aspecto	Parámetro	Atributo
Origen	URL o formulario	Código del servidor
Lectura	request.getParameter	request.getAttribute o EL
Tipo inicial	String o null	Cualquier objeto
Ejemplo conceptual	Entrada escrita por el usuario	Dato preparado para la vista

El controller puede leer un parámetro, validarlo y convertirlo en un atributo. La JSP debería trabajar con el resultado preparado, no repetir la validación de la entrada.

7 Práctica 3 Perfil dinámico

Objetivo: representar cinco atributos enviados por un controller. La interfaz ya contiene avatar, tarjeta y composición responsive.

```
String nombre = request.getParameter("nombre");
if (nombre == null || nombre.isBlank()) nombre = "Ana";

// TODO: añade al request los datos del perfil.
// TODO: realiza forward a la vista de perfil.
```

TODO	Tarea y pista
1	Muestra el rol del perfil en la insignia.
2	Muestra el nombre principal.
3	Muestra la edad y conserva el texto que acompaña al número.
4	Muestra el correo recibido.
5	Muestra el grupo del usuario.

Pruebas. Ejecuta la ruta sin parámetro y con ?nombre=Lucía. Después cambia deliberadamente una clave en setAttribute y explica por qué una parte de la vista queda vacía.

8 Expression Language y objetos

EL accede a propiedades mediante getters. La expresión ${usuario.nombre} solicita conceptualmente usuario.getNombre(). También puede navegar por una relación entre objetos.

```
${pedido.numero}
${pedido.cliente.nombre}
```

En el segundo ejemplo se obtiene primero el cliente asociado al pedido y después su nombre. Si el objeto relacionado es null, la vista no dispone de la información necesaria. Revisa entonces el DAO y el métodos privados de mapeo antes de modificar la JSP.

9 Práctica 4 Ficha visual de libro

Objetivo: recibir un Libro completo y representar sus propiedades en una ficha editorial.

```
@WebServlet("/libro")
public class LibroDetalleServlet extends HttpServlet {
    // TODO: inicializa el DAO.
    // TODO: lee y valida el parámetro id.
    // TODO: busca el libro.
    // TODO: trata los casos incorrectos.
    // TODO: publica el objeto y realiza forward.
}
```

TODO	Tarea y pista
1	Muestra la categoría asociada al libro.
2	Muestra el título principal.
3	Muestra el nombre del autor relacionado.
4	Muestra el ISBN.
5	Muestra el año.
6	Muestra el precio y conserva el símbolo de moneda.
7	Construye el parámetro id del enlace Editar.

Pruebas. Comprueba un id válido, uno inexistente y un texto no numérico. El controller debe impedir el forward cuando ya ha enviado un error.

10 JSTL sin código Java en la vista

Los scriptlets <% ... %> permiten escribir Java dentro de una JSP, pero vuelven a mezclar presentación y lógica. JSTL ofrece etiquetas de vista para recorrer colecciones y comprobar estados.

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<c:forEach var="producto" items="${productos}">
    <p>${producto.nombre}</p>
</c:forEach>
```

Parte	Significado
items	Colección que se recorrerá.
var	Nombre temporal del elemento actual.
Cuerpo de la etiqueta	Fragmento que se repite para cada elemento.

Estado vacío con c if

```
<c:if test="${empty productos}">
    ... interfaz de estado vacío ...
</c:if>
```

empty es verdadero si el valor es null o si la colección está vacía. La vista representa ese estado; no decide por qué no existen resultados.

11 Práctica 5 Biblioteca visual

Objetivo: representar una lista de libros mediante tarjetas y mostrar un estado vacío cuando no existan resultados.

```
List<Libro> libros = libroDAO.findAll();
request.setAttribute("libros", libros);
// TODO: realiza forward a la vista de listado.
```

TODO	Tarea y pista
1	Asigna un nombre a la variable que representará cada elemento.
2	Indica qué atributo del request contiene la colección.
3	Muestra la categoría del elemento actual.
4	Muestra su título.
5	Muestra el nombre del autor relacionado.
6	Muestra el año.
7	Añade el identificador al enlace de detalle.
8	Escribe la condición que muestra la cuadrícula cuando hay datos.
9	Escribe la condición complementaria para el estado vacío.

Pruebas. Ejecuta primero con varios libros y después con una lista vacía. En ambos casos la interfaz debe quedar completa. Usa la pestaña Network para comprobar el id enviado por el enlace Ver.

12 WEB INF

```
src/main/webapp/
└─ WEB-INF/
   └─ views/
      ├─ libros.jsp
      ├─ libro-detalle.jsp
      └─ libro-formulario.jsp
```

Las JSP situadas en WEB-INF no pueden solicitarse directamente desde el navegador. El usuario entra por el controller, que valida la petición y prepara los datos antes de realizar forward.

```
GET /libros -> controller -> atributos -> forward -> WEB-INF/views/libros.jsp
```

13 Forward y redirect

Aspecto	forward	redirect
Quién continúa	Tomcat	El navegador
Peticiones HTTP	Una	Dos
URL visible	No cambia	Cambia
Request	Se conserva	Se crea otro
Atributos	Siguen disponibles	No se conservan
Uso habitual	Mostrar una JSP preparada	Continuar después de un POST

```
FORWARD
Petición -> controller -> JSP -> respuesta

REDIRECT
Petición 1 -> controller -> respuesta 302
Petición 2 -> nueva ruta -> respuesta
```

14 MVC

```
PETICIÓN
   ↓
CONTROLLER  package controller
   ↓
MODEL  clases DAO métodos privados de mapeo BBDD
   ↓
CONTROLLER  prepara atributos y elige vista
   ↓
VIEW  JSP EL JSTL
   ↓
RESPUESTA HTML
```

Componente	Responsabilidad	No debe hacer
Controller	Recibir HTTP, validar, coordinar y elegir vista.	Construir una interfaz larga.
Model	Representar dominio y resolver datos.	Conocer CSS o botones.
View	Representar datos preparados.	Ejecutar SQL o abrir conexiones.

15 Práctica 6 Formulario dinámico

Objetivo: cargar autores y categorías mediante el controller, construir opciones dinámicas y enviar un POST de alta.

```
// GET /libros/nuevo
// TODO: consulta autores y categorías.
// TODO: añade ambas colecciones al request.
// TODO: realiza forward al formulario.
```

TODO	Tarea y pista
1	Construye el destino del formulario teniendo en cuenta el context path.
2	Selecciona el método HTTP apropiado para un alta.
3	Nombra la variable que representa un autor.
4	Indica la colección de autores.
5	Usa el identificador como value de cada option.
6	Muestra el nombre visible del autor.
7	Nombra la variable que representa una categoría.
8	Indica la colección de categorías.
9	Usa el identificador como value.
10	Muestra el nombre visible de la categoría.

Flujo que debes obtener

```
GET /libros/nuevo -> atributos -> forward -> formulario
POST /libros -> parámetros -> validar -> insert -> redirect
GET /libros -> consulta actualizada -> forward -> listado
```

Pregunta de comprensión. ¿Por qué el option muestra un nombre, pero envía un identificador? Relaciona la respuesta con getParameter y con la clave primaria.

16 Práctica final Biblioteca MVC

Integra las tres vistas de practica-final: listado, detalle y formulario. Reutiliza los DAO, métodos privados de mapeo y modelos ya conocidos. No es necesario implementar todavía edición ni borrado.

Petición	Trabajo que debes implementar
GET /libros	Consultar la lista, publicar el atributo y mostrar la vista.
GET /libro?id=…	Validar el id, buscar el objeto y mostrar detalle.
GET /libros/nuevo	Preparar autores y categorías para el formulario.
POST /libros	Leer parámetros, validar, insertar y redirigir.

Pasos de trabajo

1. Añade Jakarta Tags al proyecto Maven.
2. Copia assets y las tres JSP de practica-final.

3. Completa primero el listado y prueba datos/estado vacío.
4. Desarrolla el controller de detalle y prueba tres tipos de id.
5. Prepara las colecciones del formulario.
6. Completa los TODO de action, method, name y options.
7. Procesa el POST y aplica el patrón Post Redirect Get.
8. Reconstruye oralmente el recorrido de un libro desde BBDD hasta la tarjeta.

Actividades de autonomía

1. Añade editorial al detalle sin consultar desde la JSP.
2. Convierte el buscador visual en un filtro GET.
3. Añade una insignia condicional para libros recientes.
4. Muestra un contador de resultados.
5. Conserva valores del formulario cuando haya errores.
6. Añade una opción sin categoría solo si el modelo lo permite.

17 Errores frecuentes y pistas de diagnóstico

Síntoma	Qué debes comprobar
404 al mostrar la vista	Ubicación bajo WEB-INF/views y ruta del RequestDispatcher.
Una expresión aparece vacía	Coincidencia exacta entre setAttribute y EL.
JSTL no se reconoce	Dependencias Maven Jakarta y URI de taglib.
forEach no crea tarjetas	Atributo publicado, tipo colección y nombre de items.
WEB-INF devuelve 404	Entrada por la ruta del controller, no por la JSP.
Los atributos desaparecen	Comprueba si usaste redirect en lugar de forward.
Falla una propiedad relacionada	Inspecciona el objeto construido por DAO y su mapeo privado.
El Servlet no responde	package controller, mapping y despliegue.
Tomcat no arranca	Busca mappings duplicados.
POST recibe null	Compara name del campo con getParameter.
CSS o SVG no cargan	Revisa el context path en la URL del recurso.
Error después de forward	Comprueba que no escribiste antes en response.

18 Mapa mental final

```
NAVEGADOR
  | URL parámetros formulario
  v
TOMCAT crea request y response
  v
CONTROLLER
  | lee valida coordina modelo
  | setAttribute y forward
  v
JSP EN WEB-INF
  | EL lee propiedades
  | JSTL recorre y comprueba estados
  | produce HTML
  v
NAVEGADOR
```

Comprueba que puedes explicar

- Por qué el navegador nunca ejecuta JSP.
- Quién crea request y response.
- Diferencia entre parámetro y atributo.
- Cómo accede EL a una propiedad.
- Qué representa var dentro de c:forEach.
- Cuándo usar forward y cuándo redirect.
- Por qué las vistas se guardan en WEB-INF.
- Qué responsabilidad tiene cada componente MVC.
- Por qué JSP no consulta directamente el DAO.